BastionRL: Learning Defense and Resource Allocation

AI Mini Project — Assignment 2

Prakhar — B23BB1032 Prasangeet — B23CH1033

GitHub: BashionRL

October 26, 2026

1 Game Logic and Problem Setting

1.1 Objective and practical motivation

BastionRL is a tower-defense simulator in which an agent allocates limited resources to defend three King Towers against automatically generated enemy waves. It models a practical sequential resource-allocation problem: spending on immediate protection competes with investment in future income and recovery. Placement affects which routes are protected and whether structures survive long enough to repay their cost. This is a simulated use case, without a claim of deployment in a real infrastructure system.

The learned controller chooses defense construction, resource construction, reconstruction, or waiting. Enemy generation, movement, aiming, and projectile physics are environment dynamics; the agent does not generate an army.

1.2 Map, routes, and wave progression

The current Tiled map is 2048 × 1280 pixels, with 46 defense slots, six resource slots, and three King Tower locations. Soldiers use two routes and tanks use six routes. Seeded randomness controls enemy selection and route assignment. Baseline episodes start with 200 resources and require clearing ten waves; enemy health has no additional wave-dependent scaling.

Wave one contains five enemies. Counts grow approximately exponentially: 5, 7, 10, 14, 20, 27, 38, 53, 74, and 104. Spawns are separated by one second, with 15 seconds of initial preparation and ten seconds between waves. Walking and advanced soldiers are initially available; light tanks unlock at wave two and heavy tanks at wave three. Stronger-type probabilities increase with progression. Enemy behavior is fixed during learning.

1.3 Combat and structure destruction

Soldiers follow their route and make one suicide attack when crossing a King Tower slot. They are removed afterward, without granting a kill reward. Tanks continue moving while their independent turret fires projectiles at the nearest living structure in range. Their effective range is at least 300 pixels. Equal-distance targets follow deterministic structure-list order: defense towers, King Towers, then resource towers.

Defense towers automatically attack nearby living enemies within range. Damage uses a shared structure-health API. At zero health, defense towers stop firing, resource towers stop producing, and their slots become available for rebuilding. Fired projectiles can outlive their shooter; projectiles targeting destroyed or removed entities are discarded. Kills, destruction, and damage are accounted for once. Route departures are telemetry rather than a separate base-health loss rule.

1.4 Economy and tower choices

Structure	Cost	HP	Range
Cannon	100	300	240
Double Cannon	150	400	280
Resource Tower	50	250	—
King Tower	—	500	—

A Cannon deals 25 damage per projectile. A Double Cannon fires two-projectile bursts, dealing 20 damage per projectile. A living Resource Tower produces 50 resources every ten seconds; passive income adds 50 every 15 seconds. Defeated enemies also grant configured currency rewards. Currency income and the numerical RL reward are separate quantities.

1.5 Destroyed-only King reconstruction

Each King has 500 HP, giving an initial combined maximum of 1,500 HP. A King can be reconstructed **only after destruction**, while another King survives and the episode is running. Reconstruction costs 50 resources once and restores health gradually over eight seconds. Living damaged Kings cannot start repairs. Insufficient funds, unknown IDs, an existing repair, and terminal episodes are rejected. Incoming lethal damage cancels repair.

The final King's destruction immediately ends the episode in defeat. One or two destroyed Kings do not end it. Defeat cannot be reversed by repair or overwritten by later events. Victory requires clearing all configured waves with a surviving King. Reported base health is the sum of King HP.

1.6 Shared simulation and reproducibility

A fixed 1/60-second simulation powers both the headless Gymnasium environment and the Pygame frontend. Decisions normally advance 30 physics ticks (0.5 seconds). When waiting is the only legal choice, the environment condenses unavoidable waits, checking eligibility every 30 ticks. A 2,400-unit horizon caps the episode at 1,200 simulated seconds, including condensed waits. Victory/defeat sets `terminated`; reaching the horizon without either sets `truncated`.

Tests cover space validity, seeded determinism, damage and projectiles, invalid actions, reconstruction eligibility/cost/progress, terminal conditions, and PPO training/evaluation smoke runs.

2 RL Formulation, Training, and Results

2.1 State representation and action space

The policy observes $o_t \in \mathcal{S}_{\text{obs}} \subseteq [0,1]^{1462}$ (float32, schema 7). Enemy truncation and combat summaries hide parts of the simulator state.

Observation block	Values
Global health, economy, progression, timers	15
Two defense-type configurations	2×6
Four enemy-type configurations	4×7
Defense slots: positions, occupancy, threats	46×12
Resource slots: health, income, route proximity	6×10
Kings: health, repair state, incoming threats	3×9
Active enemies: fixed-size padded representation	64×12
Total	1,462

Enemy rows contain presence, position, health fraction, type, route progress, maximum health, speed, damage, range, targeting status, and cooldown. Rows retain the 64 most advanced enemies by route progress, with zero-padding and overflow telemetry. Positions are map-normalized; unbounded nonnegative quantities use $x/(x+c)$.

The discrete action space $\mathcal{A} = \{0, \ldots, 101\}$ contains $1 + 46 \times 2 + 6 + 3 = 102$ actions: wait, build either defense type in a slot, build a Resource Tower, or reconstruct a King. Masks enforce affordability, occupancy, and repair eligibility. Invalid requests are rejected and penalized while advancing time. Masks do not prescribe tactics.

2.2 Reward and learning rule

For one recorded transition, the raw reward is

$$\begin{aligned} r_t = {} & 0.05 D_t + 0.025 K_t + 0.1 W_t - \frac{B_t}{1500} \\ & - 0.001 \frac{C_t}{100} - 0.005 I_t \\ & + 2.51 \mathbf{1}_{\text{win}} - \mathbf{1}_{\text{loss or timeout}}. \end{aligned}$$

Here D_t sums actual enemy damage divided by each victim's maximum HP; K_t counts kills, W_t completed waves, B_t actual King damage, C_t spending, and I_t an invalid-action indicator. Income, healing, and waiting earn no reward. Damage after reconstruction is penalized again. Combat shaping supplies intermediate credit; spending discourages waste.

We use SB3-Contrib **Maskable PPO**, with a learned hierarchical action distribution and separate actor/critic MLPs, each with two 128-unit layers:

$$\pi_\theta(a \mid o) = \pi_\theta(k \mid o)\, \pi_\theta(u \mid k, o).$$

Here k is wait/defense/resource/reconstruction and u its concrete action. PPO optimizes joint log probabilities with a clipped objective [1], value loss, and entropy regularization. Evaluation selects the highest-probability legal kind, then its highest-probability target. This avoids comparing wait against defense probability fragmented across individual slots. The submitted agent uses no demonstrations, heuristic search, or scripted tactical fallback.

2.3 Training protocol

Seed-0 training uses eight subprocess CPU environments, 512 transitions each (4,096/rollout), batch size 256, and five epochs. PPO uses $\gamma = 0.999$, GAE $\lambda = 0.98$, clip 0.2, and target KL 0.03. Across the requested two-million-transition budget, linear schedules decay learning rate $3 \times 10^{-4} \to 5 \times 10^{-5}$ and entropy coefficient $0.02 \to 0.005$. VecNormalize standardizes observations/rewards; evaluation freezes statistics and reports raw returns. Discounting is per recorded transition, including condensed waits.

Mixed curriculum samples approximately 60% current wave goal, 20% easier goals, and 20% full ten-wave games. Promotion requires 30/50 recent current-goal successes; observations include the goal. Full-game validation uses seeds 10000–10029 every 65,536 transitions. Checkpoints rank by wins, waves completed, then King HP.

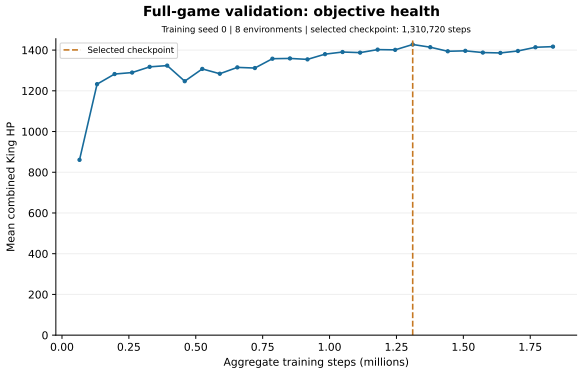


Figure 1: Full-game validation King HP (30 games/checkpoint). Dashed line: selected step 1,310,720.

2.4 Held-out performance and limitations

On held-out baseline seeds 90000–90099, separate from validation, the selected checkpoint averaged 10 completed waves, 1401.35/1500 King HP, 349.65 kills, and 2.35 departures per episode. Raw episode return was 29.512 ± 0.098 (standard deviation). There were no timeouts. Waiting occurred in 2.55% of decisions when another action was legal.

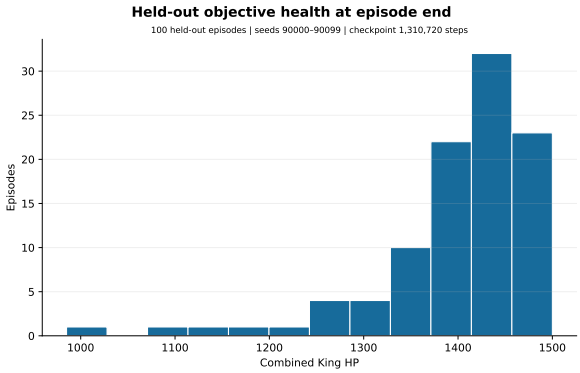


Figure 2: Final King HP across 100 held-out baseline games.

These results cover one training seed and one map. Medium/hard difficulty, unseen-map transfer, and a matched-budget flat-PPO comparison remain unmeasured. No repairs were legal in these games, so learned reconstruction is unverified. The selected model has 1,310,720 transitions; full two-million-step completion is unverified.

References

[1] J. Schulman et al., "Proximal Policy Optimization Algorithms," 2017. arXiv:1707.06347.